Lab Report-5

1. Implementation of Knapsack and Longest Common Subsequence (LCS) Algorithms.

Objectives

- To implement the Knapsack Algorithm using Dynamic Programming and determine the highest possible profit that can be achieved within a fixed knapsack capacity.
- To create a program for the Longest Common Subsequence (LCS) Algorithm that identifies the longest matching subsequence between two given strings.
- To gain practical experience with Dynamic Programming by solving optimization and sequence-matching problems.
- To evaluate the effectiveness of Dynamic Programming in minimizing unnecessary computations and improving algorithmic efficiency.

1. Code of Knapsack:

New-Semester-2.2 > Algorithm_Lab_Report > knapsack.cpp > knapsack(int, float [], float [], floa

```cpp
#include<iostream>
using namespace std;
void knapsack(int n, float weight[], float profit[], float capacity){
    float x[20], tp = 0;
    int i, j;
    float u;
    u = capacity;
    for (i = 0; i < n; i++){
        x[i] = 0.0;
    }
    for (i = 0; i < n; i++){
        if (weight[i] > u){
            break;
        }
        else{
            x[i] = 1.0;
            tp = tp + profit[i];
            u = u - weight[i];
        }
    }
    if (i < n){
        x[i] = u / weight[i];
        tp = tp + (x[i] * profit[i]);
    }
    cout<<"\nThe result vector is: ";
    for(i = 0; i < n; i++){
        cout << x[i] << "\t";
    }
    cout << "\nMaximum profit is: " << tp;
}
int main(){
    float weight[20], profit[20], ratio[20];
    float capacity, temp;
    int num, i, j;
    cout << "Enter the number of objects: ";
```

New-Semester-2.2 > Algorithm_Lab_Report > knapsack.cpp > knaps

```cpp
    cin >> num;
    cout << "Enter weight and profit of each object:\n";
    for (i = 0; i < num; i++){
        cin >> weight[i] >> profit[i];
    }
    cout << "Enter the capacity of knapsack: ";
    cin >> capacity;
    for (i = 0; i < num; i++){
        ratio[i] = profit[i] / weight[i];
    }
    for (i = 0; i < num; i++){
        for (j = i + 1; j < num; j++){
            if (ratio[i] < ratio[j]){
                temp = ratio[i];
                ratio[i] = ratio[j];
                ratio[j] = temp;
                temp = weight[i];
                weight[i] = weight[j];
                weight[j] = temp;
                temp = profit[i];
                profit[i] = profit[j];
                profit[j] = temp;
            }
        }
    }
    knapsack(num, weight, profit, capacity);
    return 0;
}
```

Output:

```
ER/New-Semester-2.2/Algorithm_Lab_Report/"kr
Enter the number of objects: 3
Enter weight and profit of each object:
18 25
15 25
10 15
Enter the capacity of knapsack: 20

The result vector is: 1 0.5 0
Maximum profit is: 32.5
macbookairm5@KAISER Algorithm_Lab_Report %
```

2. Code of LCS:

ew-Semester-2.2 > Algorithm_Lab_Report > longest_sequence.cpp > lcs()

```cpp
#include<iostream>
#include<cstring>
using namespace std;
#define MAX 100
int m, n;
int c[MAX][MAX];
char x[MAX], y[MAX];
char b[MAX][MAX];
void printLCS(int i, int j){
    if (i == 0 || j == 0){
        return;
    }
    if(b[i][j] == 'c'){
        printLCS(i - 1, j - 1);
        cout << x[i - 1];
    }
    else if(b[i][j] == 'u'){
        printLCS(i - 1, j);
    }
    else{
        printLCS(i, j - 1);
    }
}
void lcs(){
    m = strlen(x);
    n = strlen(y);
    for(int i = 0; i <= m; i++){
        c[i][0] = 0;
    }
    for(int j = 0; j <= n; j++){
        c[0][j] = 0;
    }

    for(int i = 1; i <= m; i++){
        for (int j = 1; j <= n; j++){
            if(x[i - 1] == y[j - 1]){
                c[i][j] = c[i - 1][j - 1] + 1;
                b[i][j] = 'c';
            }
            else if(c[i - 1][j] >= c[i][j - 1]){
                c[i][j] = c[i - 1][j];
                b[i][j] = 'u';
            }
```

New-Semester-2.2 > Algorithm_Lab_Report > longest_sequence.cpp

```cpp
void lcs(){
    for(int i = 1; i <= m; i++){
        for (int j = 1; j <= n; j++){
            else if(c[i - 1][j] >= c[i][j - 1]){…
            else{
                c[i][j] = c[i][j - 1];
                b[i][j] = 'l';
            }
        }
    }
}
int main(){
    cout << "Enter the 1st sequence: ";
    cin >> x;
    cout << "Enter the 2nd sequence: ";
    cin >> y;
    lcs();
    cout << "\nLength Table (c):\n";
    for(int i = 0; i <= m; i++){
        for(int j = 0; j <= n; j++){
            cout << c[i][j] << "\t";
        }
        cout << "\n";
    }
    cout << "\nDirection Table (b):\n";
    for(int i = 1; i <= m; i++){
        for(int j = 1; j <= n; j++){
            cout << b[i][j] << "\t";
        }
        cout << "\n";
    }
    cout << "\nLength of LCS = " << c[m][n] << endl;
    cout << "LCS = ";
    printLCS(m, n);
    cout << endl;
    return 0;
}
```

Output:

```
ab_Report/"longest_sequence
Enter the 1st sequence: BAUST
Enter the 2nd sequence: BAIUST

Length Table (c):
0       0       0       0       0       0       0
0       1       1       1       1       1       1
0       1       2       2       2       2       2
0       1       2       2       3       3       3
0       1       2       2       3       4       4
0       1       2       2       3       4       5

Direction Table (b):
c       l       l       l       l       l
u       c       l       l       l       l
u       u       u       c       l       l
u       u       u       u       c       l
u       u       u       u       u       c

Length of LCS = 5
LCS = BAUST
macbookairm5@KAISER Algorithm_Lab_Report %
```

Conclusion

The implementation of the Knapsack and Longest Common Subsequence (LCS) algorithms was completed successfully using the Dynamic Programming approach. The Knapsack algorithm effectively selected the optimal combination of items to achieve the maximum possible profit within the given capacity, while the LCS algorithm correctly identified the longest common subsequence shared by two input strings. Through this experiment, a better understanding of Dynamic Programming concepts, optimization techniques, and efficient problem-solving strategies was achieved. The results obtained from multiple test cases verified the correctness of both algorithms and demonstrated their usefulness in solving practical computational problems with improved efficiency.